

Organização de código

Aprendendo sobre módulos



Sumário



1.

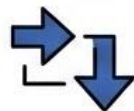
Problema que ela
(organização) resolve.



2.



O que é um módulo
em Python.



3.



Formas de importar
módulos.



4.

Pacotes.



5.

Estrutura de
projeto completa.



6.

Plus: o bloco
`__init__`

O que ela resolve?

O que ela resolve?

- começamos um projeto
- criamos um arquivo
- dentro dele criamos pequenas classes e aplicamos toda a regra nesse mesmo arquivo

No começo funciona, é ok, ***mas o que acontece conforme o projeto cresce?***

```
1 class Produto:
2     def __init__(self, nome, preco):
3         self.nome = nome
4         self.preco = preco
5
6 p1 = Produto("Teclado Mecânico", 349.90)
```

E depois...

agora preciso de uma regra envolvendo clientes, além de produtos

depois preciso de regras envolvendo pedidos

depois quero imprimir relatórios sobre os produtos vendidos

depois quero enviar e-mails para meus clientes

depois quero fazer compra para os produtos que estão com estoque baixo

aulas > 05 - organização de código > exemplos > 🐍 loja.py > ...

```
1  # main.py – 3 semanas depois
2
3  class Produto:
4  |    ... # 30 linhas
5
6  class Cliente:
7  |    ... # 40 linhas
8
9  class Pedido:
10 |    ... # 60 linhas
11
12 def calcular_frete(cep):
13 |    ... # 25 linhas
14
15 def gerar_relatorio(pedidos):
16 |    ... # 50 linhas
17
18 def enviar_email(destinatario, mensagem):
19 |    ... # 35 linhas
20
21 # código que executa tudo isso
22 clientes = [...]
23 pedidos = [...]
24 # mais 100 linhas...
```

PROBLEMAS CONCRETOS



FALTA DE REUSO

não consegue reutilizar nada de **Produto** em outro projeto sem usar o arquivo inteiro.

- Cliente, Pedido, etc, vem tudo junto, mesmo sem necessidade deles.



CONFLITOS DE COLABORAÇÃO

Dois programadores não conseguem trabalhar no mesmo arquivo sem conflitos de alterações recorrentes.



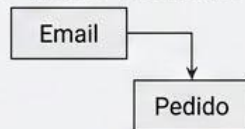
DIFICULDADE DE NAVEGAÇÃO

Dificuldade de encontrar as classes que estão dentro do mesmo arquivo.

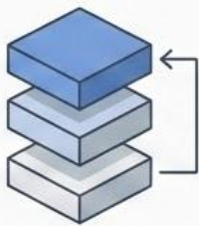


Um erro no envio de e-mails pode quebrar os pedidos (mesmo não tendo

BRITTLE/FALTA DE ISOLAMENTO
Um erro no envio de e-mails pode quebrar os pedidos.



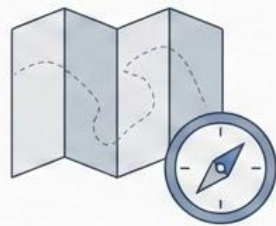
O que ela resolve



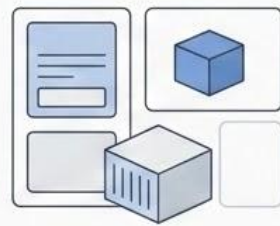
Reuso



Colaboração



Navegação



Isolamento

Módulos em Python

Conceito

- todo arquivo . py é um módulo
- nome do módulo -> nome do arquivo (sem a extensão)

[produto.py](#) -> módulo produto

[carro.py](#) -> módulo carro

[pessoa.py](#) -> módulo pessoa

```
1  import math
2
3  def calculate_circle_area(raio):
4      """Calcula a área de um círculo dado o raio."""
5      if raio < 0:
6          raise ValueError("O raio não pode ser negativo.")
7      area = math.pi * (raio ** 2)
8      return area
```

```
1  import random
2  valor = random.randint(1, 10)
3  print(valor) # Gera um número inteiro aleatório entre 1 e 10
```

math e *random* são arquivos .py que fazem parte da biblioteca padrão do python

Quando criamos um arquivo .py, estamos criando um módulo que outros arquivos podem reutilizar

Formas de importar

E quais usar em cada situação

Importar o módulo inteiro

```
1 import random
2
3 v = random.randint(1, 100)
```



REGRA MÍNIMA

Prefixo sempre obrigatório (ex: ``random.randint``). Não use direto ``randint``.



VANTAGENS

A origem de cada função chamada fica bem definida.



DESVANTAGEM

Torna o código muito verboso quando muitas funções são utilizadas.

Importar partes específicas

```
1  from random import randint
2
3  valor = randint(1, 10) # nao preciso mais do "random." antes do "randint"
```

Vantagens:

- código limpo

Desvantagens:

- se dois módulos diferentes tiverem um nome em comum, existe conflito

```
1  from conta_bancaria import sacar
2  from volei import sacar
3
4  sacar() # Qual função "sacar" será chamada? A do vôlei ou a da conta bancária?
```


Importas com “apelido”

alias.py U X

ESP

05 - organização de código > exemplos > imports > alias.py > ...

```
1 import random as r
2
3 v = r.randint(1, 100)
4 print(v)
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

ESP-IDF

POLYGLOT NOTEBOOK

...

Python - exemplo

- PS C:\workspace\Aprender e crescer\aprender-e-crescer\P00\aulas\05 - organização de código\exemplos> & ocal\Programs\Python\Python312\python.exe "c:/workspace/Aprender e crescer/aprender-e-crescer/P00/aulas/igo/exemplos/imports/alias.py"
13
- PS C:\workspace\Aprender e crescer\aprender-e-crescer\P00\aulas\05 - organização de código\exemplos> █

Importar TUDO

```
1 from random import *
```

Importa todos os nomes de dentro de um módulo.

Desvantagens:

- não sabe tudo que veio
- dificulta leitura e debug em caso de sobrescrita

Pacotes

Conceito e justificativa

Aplicações com 3/4/5 módulos, com todos os arquivos .py em uma única pasta perdem clareza. ***Pacotes resolvem isso***

Como criar um pacote?

- crie uma nova pasta, que vai ter o nome do pacote que você deseja
 - exemplo: preciso de um pacote que vai conter todas as minhas classes principais, meus “modelos” principais, então vou criar uma pasta chamada “models”
- deixe dentro dessa pasta, todos os arquivos .py que devem fazer parte daquele pacote



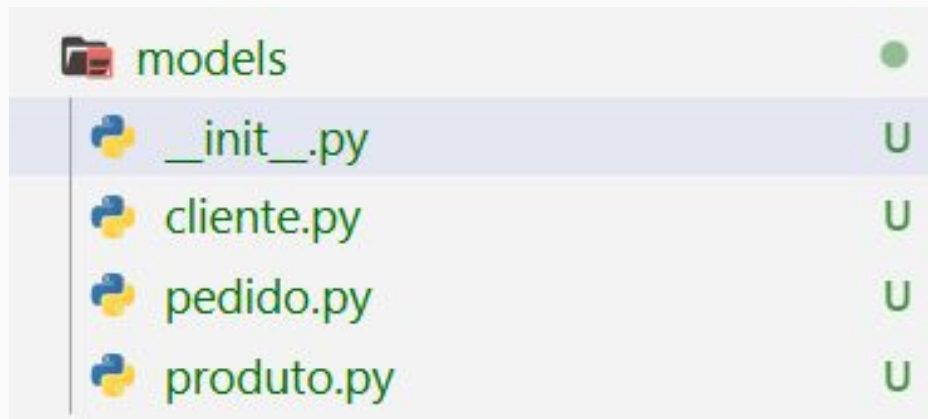
Como criar um pacote?

O que acontece quando importamos algo desse pacote, do jeito que está?

```
... from models.cliente import Cliente  
ImportError: cannot import name 'Cliente' from 'models.cliente'
```

Como criar um pacote?

- Para corrigir esse problema, precisamos criar um arquivo dentro do nosso pacote
 - `__init__.py`
 - não precisa ter nada, apenas a sua existência já sinaliza pro python que se trata de um pacote



Importando um pacote

Importando vários nomes do mesmo módulo

```
1  # from models.cliente import Cliente
2  # from models.produto import Produto
3  💡
4  from models import cliente, produto, pedido
```

Estrutura de projeto completa

```
loja/
├── modelos/
│   ├── __init__.py
│   ├── produto.py      ← class Produto
│   ├── cliente.py      ← class Cliente
│   └── pedido.py       ← class Pedido
├── servicos/
│   ├── __init__.py
│   ├── estoque.py      ← lógica de estoque
│   └── relatorio.py    ← geração de relatórios
├── testes/
│   ├── __init__.py
│   └── test_produto.py
└── main.py             ← ponto de entrada
```

Plus: o bloco `__main__`

Organização de código

Aprendendo sobre módulos

